

BloodBridge: Optimizing Lifesaving Resources using RDS,EC2

A Project Report

Submitted to Smart-Internz

for the completion of [Cloud Practitioner](#) Internship Program

Prepared By

Bilal Rafik Mirje

PRN: 2022011031111

D. Y. Patil Agriculture and Technical University, Talsande

Academic Year: 2025–2026

Submitted To Smart-Internz

BloodBridge: Optimizing Lifesaving Resources using RDS and EC2

INDEX

1. Introduction
 - 1.1 Project Overview
2. Objectives
3. Project Initialization and Planning Phase
 - 3.1 Define Problem Statement
 - 3.2 Project Proposal
 - 3.3 Initial Project Planning
4. System Design and Architecture
 - 4.1 System Architecture
 - 4.2 Technologies Used
 - 4.3 Database Design
5. AWS Cloud Setup Phase
 - 5.1 EC2 Instance Setup
 - 5.2 RDS Database Setup
 - 5.3 Database Integration
6. Web Application Development Phase
 - 6.1 User Registration and Login
 - 6.2 Role-Based Dashboard
 - 6.3 Emergency Blood Request
 - 6.4 Donor Management
 - 6.5 Blood Bank Inventory Management
7. Testing and Optimization
 - 7.1 Testing Strategy
 - 7.2 Performance Optimization
8. Results
 - 8.1 Output Screenshots
9. Advantages and Disadvantages
 - 9.1 Advantages
 - 9.2 Disadvantages
10. Conclusion
11. Future Scope
12. Appendix

1. Introduction

1.1 Project Overview

BloodBridge is a web-based blood donation and blood resource management system designed to connect donors, hospitals, blood banks, and administrators on a single platform. The main purpose of this project is to reduce the time required to find suitable blood donors during emergency situations and to improve the management of blood inventory.

The system provides different dashboards for donors, hospitals, blood banks, and administrators. Hospitals can create urgent blood requests, donors can view blood drives and manage donations, and blood banks can update blood inventory. The project is designed to work with AWS EC2 for hosting and AWS RDS MySQL for database management.

2. Objectives

The main objectives of this project are:

- To develop a web application for blood donation and blood request management.
 - To allow hospitals to raise emergency blood requests.
 - To allow donors to register, login, and manage donation activities.
 - To allow blood banks to update and manage blood inventory.
 - To provide role-based dashboards for admin, donor, hospital, and blood bank users.
 - To integrate the application with a MySQL database.
 - To make the system compatible with AWS EC2 and RDS deployment.
 - To improve the speed and efficiency of blood resource availability.
-

3. Project Initialization and Planning Phase

3.1 Define Problem Statement

In emergency medical situations, finding the required blood type quickly can be difficult. Hospitals often depend on manual communication with blood banks and donors, which may

cause delays. Blood banks also need an efficient system to update stock levels and prioritize low-availability blood types.

Therefore, there is a need for a centralized system that connects hospitals, donors, and blood banks in real time.

3.2 Project Proposal

BloodBridge proposes a cloud-compatible web application where:

- Hospitals can create emergency blood requests.
- Donors can view requests and blood drives.
- Blood banks can update blood inventory.
- Admin can monitor users, requests, and donation activities.

The system uses Flask for backend development, MySQL/RDS for database storage, and HTML/CSS templates for the user interface.

3.3 Initial Project Planning

The project was planned in the following phases:

1. Requirement analysis
 2. Database design
 3. Flask application setup
 4. User authentication implementation
 5. Dashboard creation
 6. Blood request and inventory module development
 7. Testing and debugging
 8. AWS deployment planning
-

4. System Design and Architecture

4.1 System Architecture

The system follows a web-based client-server architecture.

Users interact with the application through a browser. The Flask backend handles user requests, authentication, database operations, and dashboard routing. The database stores user details,

blood requests, blood inventory, donations, and blood drive information.

Basic flow:

User Browser → Flask Web Application → MySQL/RDS Database

4.2 Technologies Used

- Frontend: HTML, CSS, Bootstrap
- Backend: Python Flask
- Database: MySQL / AWS RDS MySQL
- ORM: Flask-SQLAlchemy
- Authentication: Flask-Login
- Cloud Hosting: AWS EC2
- Database Hosting: AWS RDS
- Connector: PyMySQL
- IDE: VS Code

4.3 Database Design

Main database tables include:

- User
- Admin
- Donor
- Hospital
- BloodBank
- BloodRequest
- BloodInventory
- BloodDrive
- Donation
- DriveRegistration

The User table stores common login information. Donor, Hospital, BloodBank, and Admin are role-based user types.

5. AWS Cloud Setup Phase

5.1 EC2 Instance Setup

An EC2 instance can be used to host the Flask web application. The application files are uploaded to the EC2 server, required Python packages are installed, and the Flask application is executed on the server.

Main EC2 setup steps:

1. Launch EC2 instance.
2. Install Python and pip.
3. Upload project files.
4. Install dependencies.
5. Configure security group for web access.
6. Run the Flask application.

5.2 RDS Database Setup

AWS RDS MySQL can be used as the database server. It provides a managed MySQL database that can be connected with the Flask application.

Main RDS setup steps:

1. Create RDS MySQL instance.
2. Configure username, password, and database name.
3. Allow EC2 security group access.
4. Copy RDS endpoint.
5. Add RDS connection string in Flask config.

5.3 Database Integration

The Flask application connects to MySQL/RDS using SQLAlchemy and PyMySQL.

Example database URI:

```
SQLALCHEMY_DATABASE_URI =  
"mysql+pymysql://username:password@host:3306/database_name"
```

For local testing, localhost MySQL can also be used.

6. Web Application Development Phase

6.1 User Registration and Login

The system provides login functionality for different users. Each user has an email, password, and user type. Passwords are stored in hashed format for security.

User roles include:

- Admin
- Donor
- Hospital
- Blood Bank

6.2 Role-Based Dashboard

After login, users are redirected to their respective dashboards based on their user type.

- Admin Dashboard: Manage overall system.
- Donor Dashboard: View blood drives and donation information.
- Hospital Dashboard: Create and manage blood requests.
- Blood Bank Dashboard: Update inventory and manage blood drives.

6.3 Emergency Blood Request

Hospitals can create emergency blood requests by entering:

- Blood type
- Units needed
- Priority level
- Patient details
- Deadline

This helps hospitals quickly raise urgent requests for required blood types.

6.4 Donor Management

Donors can register and login to the system. They can view donation-related information and

participate in blood drives. This helps maintain a steady supply of blood for hospitals and blood banks.

6.5 Blood Bank Inventory Management

Blood bank users can manage available blood stock. They can update units available for different blood types such as O+, A-, B+, and AB+.

This helps hospitals and administrators understand current blood availability.

7. Testing and Optimization

7.1 Testing Strategy

The system was tested using the following methods:

- Login testing with different user roles
- Database connection testing
- Blood request creation testing
- Blood inventory update testing
- Dashboard redirection testing
- Form validation testing
- Local MySQL testing

7.2 Performance Optimization

Performance can be improved by:

- Using indexed database columns
 - Avoiding duplicate database queries
 - Using optimized SQLAlchemy relationships
 - Hosting application on EC2
 - Using RDS for reliable database performance
-

8. Results

8.1 Output Screenshots

Screenshots can be added in this section:

- Home Page
- Login Page
- Admin Dashboard
- Donor Dashboard
- Hospital Dashboard
- Blood Bank Dashboard
- Emergency Blood Request Page
- Blood Inventory Page

Example:

```
screenshots/home.png  
screenshots/login.png  
screenshots/admin-dashboard.png  
screenshots/blood-request.png  
screenshots/inventory.png
```

9. Advantages and Disadvantages

9.1 Advantages

- Reduces time required to find blood donors.
- Provides role-based access.
- Helps hospitals create urgent blood requests.
- Helps blood banks manage inventory.
- Cloud-compatible using EC2 and RDS.
- Centralized blood resource management.
- Improves communication between hospitals, donors, and blood banks.

9.2 Disadvantages

- Internet connection is required.

- Real SMS/email notification may need additional integration.
 - Cloud deployment requires AWS configuration.
 - System depends on accurate inventory updates.
 - Security must be improved before production deployment.
-

10. Conclusion

BloodBridge is a useful web-based system for managing blood donation, emergency blood requests, and blood bank inventory. It provides separate dashboards for donors, hospitals, blood banks, and administrators. The project demonstrates how cloud technologies like AWS EC2 and RDS can be used to build a scalable and reliable healthcare support system.

This project helps improve the speed and efficiency of blood availability during emergency situations and supports better coordination between donors and healthcare organizations.

11. Future Scope

Future improvements can include:

- SMS notification system for nearby donors.
 - Email alerts for emergency blood requests.
 - Google Maps integration to find nearby donors.
 - Real-time notification using WebSocket.
 - Mobile application version.
 - AI-based donor matching system.
 - Blood expiry tracking.
 - Advanced admin analytics dashboard.
 - OTP-based login and verification.
 - Deployment with HTTPS and domain name.
-

12. Appendix

12.1 Source Code

The project source code includes:

- Flask application files

- Models
- Routes
- Templates
- Static files
- Database initialization file
- Configuration file

12.2 GitHub and Project Video Demo Link

GitHub Link:

<https://github.com/BilalMirje/BloodBridge>

Project Demo Video Link:

<https://drive.google.com/file/d/14qkKP2T19UkPAcy5amOKJ4DtA9O0z9ia/view>